COMP 442
Compiler Design

Assignment 3: Abstract Syntax Tree

Writer:
Théo Bary (ID: 40348931)

Instructor: Dr. Joey Paquet
joey.paquet@concordia.ca

Date Due: Sunday, 08 March 2026, 11:59 PM

"I certify that this submission is my original work and meets the Faculty's Expectations of Originality"

Section 1 - Grammar: 2
ATOMIC SEMANTIC CONCEPTS (AST Leaves) 2
COMPOSITE SEMANTIC CONCEPTS (AST Subtrees) 3
PROGRAM STRUCTURE 3
CLASS DECLARATIONS 4
VARIABLE DECLARATIONS 5
FUNCTION DECLARATIONS & DEFINITIONS 7
STATEMENTS 9
EXPRESSIONS 11
VARIABLE ACCESS 13
FUNCTION CALLS 14
VISIBILITY MODIFIERS 16
SPECIAL AST CONCEPTS 16
Section 2 - FIRST and FOLLOW sets: 17
Section 3 - Design: 18
Section 4 - Use of tools: 19

Section 1 - Grammar:

ATOMIC SEMANTIC CONCEPTS (AST Leaves)

1. Identifier
- AST Leaf: Id node
- Lexeme: Variable name, class name, function name
- Examples: x, count, POINT, factorial

2. Type Annotation
- AST Leaf: Type node
- Lexeme: Type name (integer, float, or user-defined class name)
- Examples: integer, float, MATRIX, VEC

3. Integer Literal
- AST Leaf: Num node with integer value
- Lexeme: Integer constant
- Examples: 0, 42, 255, -1

4. Float Literal
- AST Leaf: Num node with float value
- Lexeme: Float constant
- Examples: 0.0, 3.14, 1.5, -2.0

5. Operator Symbols
- AST Leaves: Stored as lexemes in operator nodes
- Addition/Subtraction: +, -, or
- Multiplication/Division: *, /, and
- Relational: ==, <>, <, >, <=, >=
- Unary: not, + (unary plus), - (unary minus)

COMPOSITE SEMANTIC CONCEPTS (AST Subtrees)

PROGRAM STRUCTURE

6. Program
- AST Subtree:

  Prog
  - ClassList
  - FuncDefList
  - ProgramBlock

- Description: The root of the entire program
- Components: Class declarations, free function definitions, and main program block

7. Class List
- AST Subtree:

  ClassList
  - Class
  - Class
  - ...

- Description: Collection of all class declarations
- Can be empty: Yes (no classes)

8. Function Definition List
- AST Subtree:

  FuncDefList
  - FuncDef
  - FuncDef
  - ...

- Description: Collection of all free and member function definitions
- Can be empty: Yes (no functions besides main)

9. Main Program Block
- AST Subtree:

  ProgramBlock
  - VarDecl* (local variables, 0 or more)
  - StatBlock

- Description: The main entry point with optional local variables and statements

CLASS DECLARATIONS

10. Class Declaration (No Inheritance)
- AST Subtree:

  Class
  - Id (class name)
  - ClassList (empty - for inheritance list)
  - MemberDecl* (0 or more members)

- Example: class EMPTY { };

11. Class Declaration (With Single Inheritance)
- AST Subtree:

  Class
  - Id (class name)
  - ClassList
    - Id (parent class name)
  - MemberDecl* (members)

- Example: class CHILD inherits BASE { ... };

12. Class Declaration (With Multiple Inheritance)
- AST Subtree:

```
Class
- Id (class name)
- ClassList
    - Id (parent1)
    - Id (parent2)
- MemberDecl* (members)
```

- Example: class MULTI inherits BASE, NAMED { ... };

13. Inheritance List
- AST Subtree:

```
ClassList (used as inheritance list)
- Id (parent class 1)
- Id (parent class 2)
- ...
```

- Description: List of parent classes for inheritance

VARIABLE DECLARATIONS

14. Simple Variable Declaration
- AST Subtree:

```
VarDecl
- Type
- Id (variable name)
- DimList (empty)
```

- Examples: integer count;, float x;, COLOR bg;

15. 1D Array Variable Declaration (Fixed Size)
- AST Subtree:

```
VarDecl
- Type
- Id (array name)
- DimList
  - Dim
    - Num (size)
```

- Example: integer nums[6];

16. 2D Array Variable Declaration
- AST Subtree:

```
VarDecl
- Type
- Id (array name)
- DimList
  - Dim
  │  - Num (size1)
  - Dim
    - Num (size2)
```

- Example: float matrix[3][3];, integer data[4][4];

17. Unsized Array Declaration
- AST Subtree:

```
VarDecl
- Type
- Id (array name)
- DimList
  - Dim (no children - empty dimension)
```

- Example: float buffer[];, integer data[];

18. Dimension List
- AST Subtree:

  DimList
  - Dim
  - Dim
  - ...

- Description: Represents array dimensions

19. Single Dimension
- AST Subtree:

  Dim
  - Num (optional - size, absent for unsized)

- Description: Single array dimension

FUNCTION DECLARATIONS & DEFINITIONS

20. Member Function Declaration
- AST Subtree: (Appears within Class node as member)

  FuncDecl (represented as VarDecl-like structure)
  - Type (return type, or "void")
  - Id (function name)
  - ParamList

- Example: public evaluate(float x) : float;

21. Free Function Definition
- AST Subtree:

  FuncDef
  - Type (return type or "void")
  - Id (function name)
  - ParamList
  - VarDecl* (local variables, 0 or more)
  - StatBlock

- Example: factorial(integer n) : integer do ... end

22. Member Function Definition (With Scope Resolution)
- AST Subtree:

  FuncDef
  - Type (return type)
  - Id (class name)
  - Id (function name)
  - ParamList
  - VarDecl* (local variables)
  - StatBlock

- Example: STACK::push(integer v) : void do ... end

23. Parameter List
- AST Subtree:

  ParamList
  - VarDecl (param 1)
  - VarDecl (param 2)
  - ...

- Description: Function parameters
- Can be empty: Yes (no parameters)

24. Array Parameter

- AST Subtree:

```
ParamList
- VarDecl
  - Type
  - Id (param name)
  - DimList
    - Dim (unsized)
```

- Example: In process(integer data[], integer n)

STATEMENTS

25. Statement Block
- AST Subtree:

```
StatBlock
- Statement
- Statement
- ...
```

- Description: Sequence of statements
- Can be empty: Yes

26. Assignment Statement
- AST Subtree:

```
AssignStat
- Variable (left-hand side)
- Expression (right-hand side)
```

- Example: count = 0;, arr[i] = 42;

27. If Statement (With Else)
- AST Subtree:

  IfStat
  - RelExpr (condition)
  - StatBlock (then branch)
  - StatBlock (else branch)

- Example: if (x < 0) then do ... end else do ... end;

28. If Statement (Empty Else)
- AST Subtree:

  IfStat
  - RelExpr (condition)
  - StatBlock (then branch)
  - StatBlock (empty)

- Example: if (n > 0) then do ... end else ;

29. While Statement
- AST Subtree:

  WhileStat
  - RelExpr (condition)
  - StatBlock (loop body)

- Example: while (i < n) do ... end;

30. Read Statement
- AST Subtree:

  ReadStat
  - Variable (input destination)

- Example: read(x);, read(arr[i]);

31. Write Statement
- AST Subtree:

  PutStat
  - Expression (output value)

- Example: write(count);, write(a + b);

32. Return Statement
- AST Subtree:

  ReturnStat
  - Expression (return value)

- Example: return (result);, return (x * x);

EXPRESSIONS

33. Addition/Subtraction Expression
- AST Subtree:

  AddOp (lexeme: "+", "-", or "or")
  - Expression (left operand)
  - Expression (right operand)

- Example: a + b, x - 1, a or b

34. Multiplication/Division Expression
- AST Subtree:

  MultOp (lexeme: "*", "/", or "and")
  - Expression (left operand)
  - Expression (right operand)

- Example: a * b, x / 2, a and b

35. Relational Expression
- AST Subtree:

  RelOp (lexeme: "==", "<>", "<", ">", "<=", ">=")
  - ArithExpr (left operand)
  - ArithExpr (right operand)

- Example: x < 0, a >= b, v <> 0

36. Logical NOT Expression
- AST Subtree:

  NotExpr
  - Expression (operand)

- Example: not a

37. Unary Sign Expression
- AST Subtree:

  SignExpr (lexeme: "+" or "-")
  - Expression (operand)

- Example: -x, +a, -1

38. Parenthesized Expression
- AST Representation: No explicit node; parentheses affect parsing precedence but don't appear in AST
- Example: (a + b) produces the same AST as a + b (but with different precedence context)

VARIABLE ACCESS

39. Simple Variable Reference
- AST Subtree:

  Id (variable name)

- Example: x, count

40. Array Indexing (Single Dimension)
- AST Subtree:

  IndexedVar
  - Variable (array)
  - Expression (index)

- Example: arr[i], nums[0]

41. Array Indexing (Multi-Dimensional)
- AST Subtree:

  IndexedVar
  - IndexedVar
    - Variable (array)
    - Expression (first index)
  - Expression (second index)

- Example: matrix[i][j]

42. Member Access (Field)
- AST Subtree:

  MemberAccess
  - Variable (object)
  - Id (member name)

- Example: fg.r, obj.field

43. Chained Member Access
- AST Subtree:

```
MemberAccess
- MemberAccess
    - Variable (object)
    - Id (intermediate member)
- Id (final member)
```

- Example: lst.items[i].data

44. Member Access with Array Indexing
- AST Subtree:

```
IndexedVar
- MemberAccess
    - Variable (object)
    - Id (member array name)
- Expression (index)
```

- Example: lst.items[i]

45. Complex Nested Access
- AST Subtree: Combination of MemberAccess, IndexedVar, and FuncCall
- Example: lst.items[i].data[j], v.data[offset(i, j)]

FUNCTION CALLS

46. Free Function Call (No Arguments)
- AST Subtree:

```
FuncCall
- Id (function name)
- ParamList (empty)
```

- Example: isEmpty()

47. Free Function Call (With Arguments)
- AST Subtree:

```
FuncCall
- Id (function name)
- ParamList
  - Expression (arg1)
  - Expression (arg2)
  - ...
```

- Example: factorial(n), clamp(a, 0, 10)

48. Member Function Call
- AST Subtree:

```
FuncCall
- MemberAccess
    - Variable (object)
    - Id (method name)
- ParamList
  - Expression* (arguments)
```

- Example: s.push(10), u.length()

49. Chained Function Call
- AST Subtree:

```
FuncCall
- FuncCall
    - MemberAccess
        - ...
    - ParamList
- ParamList
```

- Example: lst.head().data[i], lst.getItem(i).size()

50. Function Call as Expression Component
- AST Subtree: FuncCall node appears as child of expression operators
- Example:
  - In a + factorial(n): AddOp has Id(a) and FuncCall as children
  - In arr[offset(i, j)]: IndexedVar has FuncCall as index expression

VISIBILITY MODIFIERS

51. Public Member
- AST Representation: Likely stored as metadata/annotation on member declarations within Class node
- Example: public float x;

52. Private Member
- AST Representation: Likely stored as metadata/annotation on member declarations within Class node
- Example: private integer id;

SPECIAL AST CONCEPTS

53. Marker Node
- AST Node: Marker
- Purpose: Used internally during AST construction to mark stack positions for collecting variable numbers of children
- Does not appear in final AST

54. Empty/Epsilon Productions
- AST Representation: Empty child lists or absent subtrees
- Examples:
  - Empty class body: Class with no member children
  - Empty parameter list: ParamList with no children
  - Empty statement block: StatBlock with no children
  - No inheritance: ClassList (for inheritance) with no children

Section 2 - Attribute Grammar:

SEMANTIC ACTIONS REFERENCE

The following semantic actions are used to construct the Abstract Syntax Tree (AST) during parsing. They operate on a semantic stack that accumulates AST nodes bottom-up.

LEAF NODE CREATION ACTIONS

1. {MakeId}
- Creates an Id node from the current token's lexeme
- Effect: Push Id(lexeme) onto semantic stack
- Used for: Identifiers (variable names, class names, function names)

2. {MakeType}
- Creates a Type node from the current token's lexeme
- Effect: Push Type(lexeme) onto semantic stack
- Used for: Type annotations (integer, float, class names, void)

3. {MakeNum}
- Creates a Num node from the current token's lexeme
- Effect: Push Num(lexeme) onto semantic stack
- Used for: Integer and float literals

DIMENSION AND ARRAY ACTIONS

4. {MakeDim}
- Creates a Dim node, optionally containing a size
- Effect: If top of stack is Num, pop it and store as dimension size. Push Dim(size) onto semantic stack (size may be empty for unsized dimensions)
- Used for: Single array dimension [n] or []

5. {MakeDimList}
- Creates a DimList node by collecting all dimensions since the last marker
- Effect: Pop all nodes until marker, create DimList with these children, push DimList onto semantic stack
- Used for: Complete array dimension specification

DECLARATION ACTIONS

6. {MakeVarDecl}
- Creates a VarDecl node from type, identifier, and dimension list
- Effect: Pop DimList, Id, and Type from stack (in that order). Create VarDecl with these three children and current visibility. Push VarDecl onto semantic stack
- Used for: Variable declarations, function parameters, class members

7. {MakeParamList}
- Creates a ParamList node by collecting parameters since the last marker
- Effect: Pop all nodes until marker, create ParamList with these children, push ParamList onto semantic stack
- Used for: Function parameter lists and function call argument lists

CLASS STRUCTURE ACTIONS

8. {MakeClass}
- Creates a Class node with class name and all members since the marker
- Effect: Pop all nodes until marker (these are class members). Pop Id (class name). Create Class with Id as first child, followed by all members. Push Class onto semantic stack
- Used for: Complete class declaration

9. {MakeClassList}
- Creates a ClassList node by collecting all classes since the last marker
- Effect: Pop all nodes until marker, create ClassList with these children, push ClassList onto semantic stack
- Used for: List of all class declarations and inheritance lists

10. {SaveVisibility}
- Saves the current visibility modifier (public/private) for later use
- Effect: Store current token lexeme (public/private) in a saved variable
- Used for: Attaching visibility to class member declarations

FUNCTION STRUCTURE ACTIONS

11. {MakeFuncDef}
- Creates a FuncDef node from return type, name(s), parameters, and body
- Effect: Pop StatBlock, Type, ParamList, Id from stack (in that order). For member functions, there may be two Ids (class::function). Create FuncDef with four children: Type, Id, ParamList, StatBlock. Push FuncDef onto semantic stack
- Used for: Complete function definition

12. {MakeFuncDefList}
- Creates a FuncDefList node by collecting functions since the last marker
- Effect: Pop all nodes until marker, create FuncDefList with these children, push FuncDefList onto semantic stack
- Used for: List of all function definitions

13. {MakeFuncCall}
- Creates a FuncCall node from identifier/expression and parameter list
- Effect: Pop ParamList and base expression from stack. Create FuncCall with two children: base, ParamList. Push FuncCall onto semantic stack
- Used for: Function calls and method calls

STATEMENT ACTIONS

14. {MakeStatBlock}
- Creates a StatBlock node by collecting statements since the last marker
- Effect: Pop all nodes until marker, create StatBlock with these children, push StatBlock onto semantic stack
- Used for: Statement blocks (do...end), function bodies

15. {MakeAssignStat}
- Creates an AssignStat node from left-hand side and right-hand side
- Effect: Pop rhs expression, pop lhs variable. Create AssignStat with two children: lhs, rhs. Push AssignStat onto semantic stack
- Used for: Assignment statements (variable = expression)

16. {MakeIfStat}
- Creates an IfStat node from condition, then-block, and else-block
- Effect: Pop elseBlock, thenBlock, condition from stack (in that order). Create IfStat with three children: condition, thenBlock, elseBlock. Push IfStat onto semantic stack
- Used for: If-then-else statements

17. {MakeWhileStat}
- Creates a WhileStat node from condition and loop body
- Effect: Pop body, pop condition. Create WhileStat with two children: condition, body. Push WhileStat onto semantic stack
- Used for: While loop statements

18. {MakeReadStat}
- Creates a ReadStat node from input variable
- Effect: Pop variable from stack. Create ReadStat with one child: variable. Push ReadStat onto semantic stack
- Used for: Read (input) statements

19. {MakePutStat}
- Creates a PutStat node from output expression
- Effect: Pop expression from stack. Create PutStat with one child: expression. Push PutStat onto semantic stack
- Used for: Write (output) statements

20. {MakeReturnStat}
- Creates a ReturnStat node from return expression
- Effect: Pop expression from stack. Create ReturnStat with one child: expression. Push ReturnStat onto semantic stack
- Used for: Return statements

EXPRESSION ACTIONS

21. {MakeAddOp}
- Creates an AddOp node from left operand, right operand, and saved operator
- Effect: Pop right operand, pop left operand. Create AddOp with saved operator lexeme (+, -, or). AddOp has two children: left, right. Push AddOp onto semantic stack
- Used for: Addition, subtraction, logical OR operations

22. {MakeMultOp}
- Creates a MultOp node from left operand, right operand, and saved operator
- Effect: Pop right operand, pop left operand. Create MultOp with saved operator lexeme (*, /, and). MultOp has two children: left, right. Push MultOp onto semantic stack
- Used for: Multiplication, division, logical AND operations

23. {MakeRelOp}
- Creates a RelOp node from left operand, right operand, and saved operator
- Effect: Pop right operand, pop left operand. Create RelOp with saved operator lexeme (==, <>, <, >, <=, >=). RelOp has two children: left, right. Push RelOp onto semantic stack
- Used for: Relational comparison operations

24. {MakeNotExpr}
- Creates a NotExpr node from a single operand
- Effect: Pop operand from stack. Create NotExpr with one child: operand. Push NotExpr onto semantic stack
- Used for: Logical NOT operation

25. {MakeSignExpr}
- Creates a SignExpr node from a single operand and current sign token
- Effect: Pop operand from stack. Create SignExpr with current token lexeme (+ or -). SignExpr has one child: operand. Push SignExpr onto semantic stack
- Used for: Unary plus and unary minus operations

26. {SaveOp}
- Saves the current operator token for later use
- Effect: Store current token lexeme in a saved variable
- Used for: Remembering operators for AddOp, MultOp, RelOp construction

VARIABLE ACCESS ACTIONS

27. {MakeIndexedVar}
- Creates an IndexedVar node for array indexing
- Effect: Pop index expression, pop base variable/expression. Create IndexedVar with two children: base, index. Push IndexedVar onto semantic stack
- Used for: Array element access (arr[i], matrix[i][j])

28. {MakeMemberAccess}
- Creates a MemberAccess node for member field/method access
- Effect: Pop member Id, pop object expression. Create MemberAccess with two children: object, member. Push MemberAccess onto semantic stack
- Used for: Dot notation access (obj.field, obj.method)

PROGRAM STRUCTURE ACTIONS

29. {MakeProgramBlock}
- Converts the top StatBlock into a ProgramBlock
- Effect: Change the kind of the top StatBlock node to ProgramBlock
- Used for: Main program body

30. {MakeProg}
- Creates the root Prog node of the entire AST
- Effect: Pop ProgramBlock, FuncDefList, ClassList from stack. Create Prog with three children: ClassList, FuncDefList, ProgramBlock. Push Prog onto semantic stack and save as AST root
- Used for: Complete program

UTILITY ACTIONS

31. {PushMarker}
- Pushes a marker onto the semantic stack
- Effect: Push a special Marker node onto semantic stack
- Used for: Marking the start of a collection (for collecting variable numbers of children: classes, functions, statements, parameters, dimensions, etc.)

ANNOTATED ATTRIBUTE GRAMMAR

The following grammar shows the complete syntax with embedded semantic actions indicated at their precise positions in the right-hand sides.

PROGRAM STRUCTURE

<START> ::= <prog>

<prog> ::= {PushMarker} <classList> {MakeClassList} {PushMarker} <funcDefList> {MakeFuncDefList} MAIN <funcBody> {MakeProgramBlock} {MakeProg}

<classList> ::= <classDecl> <classList>
| EPSILON

<funcDefList> ::= <funcDef> <funcDefList>
| EPSILON

CLASS DECLARATIONS

<classDecl> ::= CLASS ID {MakeId} <classInheritOpt> OPEN_BRACE {PushMarker} <classMemberList> CLOSE_BRACE SEMICOLON {MakeClass}

<classInheritOpt> ::= INHERITS ID <classInheritTail>
| EPSILON

<classInheritTail> ::= COMMA ID <classInheritTail>
| EPSILON

<classMemberList> ::= <visibility> {SaveVisibility} <memberDecl> <classMemberList>
| EPSILON

<visibility> ::= PUBLIC
| PRIVATE

<memberDecl> ::= <type_no_id> {MakeType} ID {MakeId} {PushMarker} <varArrayList> {MakeDimList} SEMICOLON {MakeVarDecl}
| ID {MakeType} <memberAfterId>

<memberAfterId> ::= ID {MakeId} {PushMarker} <varArrayList> {MakeDimList} SEMICOLON {MakeVarDecl}
| OPEN_PARENTHESIS <fParams> CLOSE_PARENTHESIS COLON <funcDeclTail>

<funcDeclTail> ::= <type> SEMICOLON
| VOID SEMICOLON

VARIABLE DECLARATIONS

<varDecl> ::= <type> {MakeType} ID {MakeId} {PushMarker} <varArrayList> {MakeDimList} SEMICOLON {MakeVarDecl}

<varDeclList> ::= <varDecl> <varDeclList>
| EPSILON

<varArrayList> ::= <arraySize> <varArrayList>
| EPSILON

<arraySize> ::= OPEN_BRACKET <arraySizeTail>

<arraySizeTail> ::= INT_NUM {MakeNum} CLOSE_BRACKET {MakeDim}
| CLOSE_BRACKET {MakeDim}

FUNCTION DEFINITIONS

<funcDef> ::= <funcHead> <funcBody> {MakeFuncDef}

<funcHead> ::= ID {MakeId} <funcHeadTail>

<funcHeadTail> ::= DOUBLE_COLON ID OPEN_PARENTHESIS {PushMarker} <fParams> CLOSE_PARENTHESIS {MakeParamList} COLON <funcHeadReturn>
| OPEN_PARENTHESIS {PushMarker} <fParams> CLOSE_PARENTHESIS {MakeParamList} COLON <funcHeadReturn>

<funcHeadReturn> ::= <type> {MakeType}
| VOID {MakeType}

<funcBody> ::= {PushMarker} <localDeclOpt> DO <stmtList> END {MakeStatBlock}

<localDeclOpt> ::= LOCAL <varDeclList>
| EPSILON

FUNCTION PARAMETERS

<fParams> ::= <type> {MakeType} ID {MakeId} {PushMarker} <fParamsArrayList> {MakeDimList} {MakeVarDecl} <fParamsTail>
| EPSILON

<fParamsTail> ::= COMMA <type> {MakeType} ID {MakeId} {PushMarker} <fParamsArrayList> {MakeDimList} {MakeVarDecl} <fParamsTail>
| EPSILON

<fParamsArrayList> ::= <arraySize> <fParamsArrayList>
| EPSILON

<aParams> ::= <expr> <aParamsTail>
| EPSILON

<aParamsTail> ::= COMMA <expr> <aParamsTail>
| EPSILON

STATEMENTS

<stmtList> ::= <statement> <stmtList>
| EPSILON

<statBlock> ::= DO {PushMarker} <stmtList> END {MakeStatBlock}
| {PushMarker} <statement> {MakeStatBlock}
| {PushMarker} {MakeStatBlock}

<statement> ::= ID {MakeId} <postfixList> <statementEnd>
| IF OPEN_PARENTHESIS <expr> CLOSE_PARENTHESIS THEN <statBlock> ELSE <statBlock> SEMICOLON {MakeIfStat}
| WHILE OPEN_PARENTHESIS <expr> CLOSE_PARENTHESIS <statBlock> SEMICOLON {MakeWhileStat}

| READ OPEN_PARENTHESIS <variable> CLOSE_PARENTHESIS SEMICOLON {MakeReadStat}
| WRITE OPEN_PARENTHESIS <expr> CLOSE_PARENTHESIS SEMICOLON {MakePutStat}
| RETURN OPEN_PARENTHESIS <expr> CLOSE_PARENTHESIS SEMICOLON {MakeReturnStat}

<statementEnd> ::= <assignOp> <expr> SEMICOLON {MakeAssignStat}
| SEMICOLON

<assignOp> ::= ASSIGN

EXPRESSIONS

<expr> ::= <arithExpr> <exprRelTail>

<exprRelTail> ::= <relOp> {SaveOp} <arithExpr> {MakeRelOp}
| EPSILON

<arithExpr> ::= <term> <arithExprTail>

<arithExprTail> ::= <addOp> {SaveOp} <term> <arithExprTail> {MakeAddOp}
| EPSILON

<term> ::= <factor> <termTail>

<termTail> ::= <multOp> {SaveOp} <factor> <termTail> {MakeMultOp}
| EPSILON

<factor> ::= ID {MakeId} <postfixList>
| INT_NUM {MakeNum}
| FLOAT_NUM {MakeNum}
| OPEN_PARENTHESIS <arithExpr> CLOSE_PARENTHESIS
| NOT <factor> {MakeNotExpr}
| <sign> <factor> {MakeSignExpr}

VARIABLE ACCESS AND FUNCTION CALLS

```
<variable> ::= ID {MakeId} <postfixListNoCall>

<postfixList> ::= <postfix> <postfixList>
            | EPSILON

<postfix> ::= OPEN_PARENTHESIS {PushMarker} <aParams> CLOSE_PARENTHESIS {MakeParamList} {MakeFuncCall}
         | OPEN_BRACKET <arithExpr> CLOSE_BRACKET {MakeIndexedVar}
         | DOT ID {MakeId} {MakeMemberAccess}

<postfixListNoCall> ::= <postfixNoCall> <postfixListNoCall>
                  | EPSILON

<postfixNoCall> ::= OPEN_BRACKET <arithExpr> CLOSE_BRACKET {MakeIndexedVar}
               | DOT ID {MakeId} {MakeMemberAccess}

<indice> ::= OPEN_BRACKET <arithExpr> CLOSE_BRACKET
```

OPERATORS

```
<relOp> ::= EQUAL
        | NOT_EQUAL
        | LESS_THAN
        | GREATER_THAN
        | LESS_EQUAL
        | GREATER_EQUAL

<addOp> ::= PLUS
        | MINUS
        | OR

<multOp> ::= MULTIPLY
         | DIVIDE
         | AND
```

<sign> ::= PLUS
| MINUS

TYPES

<type> ::= INTEGER
| FLOAT
| ID

<type_no_id> ::= INTEGER
| FLOAT

AST CONSTRUCTION STRATEGY

The semantic actions work together to build the AST using a bottom-up approach with a semantic stack:

1. LEAF CREATION
When terminals are matched (identifiers, types, numbers), semantic actions create leaf nodes and push them onto the stack.

2. MARKER-BASED COLLECTION
For constructs with variable numbers of children (lists of classes, functions, statements, parameters, dimensions), a marker is pushed before parsing the list. When the list is complete, a collection action pops all nodes until the marker and creates a parent node with all collected nodes as children.

3. FIXED-ARITY CONSTRUCTION
For constructs with fixed numbers of children (binary operators, if statements, function definitions), actions pop the exact number of required children and create a parent node.

4. SAVED STATE
Some information (operators, visibility) is saved in temporary variables during parsing and used later when constructing nodes.

5. POSTFIX OPERATIONS
Array indexing, member access, and function calls use postfix notation in the grammar. Each postfix operation modifies the base expression by wrapping it in a new node.

6. TREE ASSEMBLY

The final MakeProg action assembles the complete AST by combining the class list, function definition list, and program block into a root Prog node.

EXAMPLE: PARSING "a = b + c"

Token Stream: ID(a) ASSIGN ID(b) PLUS ID(c) SEMICOLON

Semantic Actions Executed:

1. {MakeId} -> Push Id("a")
2. {MakeId} -> Push Id("b")
3. {SaveOp} -> Save "+"
4. {MakeId} -> Push Id("c")
5. {MakeAddOp} -> Pop Id("c"), Pop Id("b"), Push AddOp("+", [Id("b"), Id("c")])
6. {MakeAssignStat} -> Pop AddOp(...), Pop Id("a"), Push AssignStat([Id("a"), AddOp(...)])

Final AST Fragment:

```
AssignStat
- Id("a")
- AddOp("+")
  - Id("b")
  - Id("c")
```

Section 3 - Design:

The AST generation is built on a table-driven LL(1) predictive parser that uses a unified grammar containing terminals, non-terminals, and semantic action symbols. Grammar productions may include embedded semantic actions treated as special parsing symbols. When the parser encounters one, it executes the action immediately, manipulating the semantic stack used to build the AST bottom-up. Because these actions are integrated directly into the grammar, AST construction occurs incrementally during parsing, removing the need for a separate tree-building phase.

AST construction relies on a semantic stack that accumulates nodes as parsing progresses. Two strategies are used: marker-based collection for variable-length constructs and fixed popping for constructs with known numbers of children. For lists (e.g., classes, parameters, statements, array dimensions), the parser pushes a marker and later gathers all nodes above it. For fixed structures (e.g., binary operators, if statements, function definitions), actions pop a set number of nodes and combine them into a parent. Auxiliary variables temporarily store information such as the current operator (SaveOp) or visibility modifier (SaveVisibility) until the appropriate node is created.

The AST node design is simple and uniform: a single `ASTNode` structure with a `Kind` enum for node type, a `lexeme` string for identifiers or operators, and a list of child nodes. This avoids complex class hierarchies while representing all constructs through node kinds and child arrangements. Nested expressions such as chained member access (obj.field.method()) or multidimensional indexing (arr[i][j]) are handled through postfix operations that repeatedly wrap the base expression in new nodes. The process ends with the MakeProg action, which combines the class list, function definitions, and program block into a single Prog root node representing the complete AST.

Section 4 - Use of tools:

For the output and debugging purposes, I chose to output my final AST to the digraph dot file format. It is relatively easy to implement and gives me the ability to put the final output into a website like Graphviz Online to visualize my result.